

Instituto Politécnico Nacional
"La Técnica al Servicio de la Patria"



Escuela Superior de Cómputo

PSO APLICADO AL MODELO HP (HIDROFÓBICO-POLAR)

Práctica 4

Alcántara López Wendy Yuritzi
Robles Guzmán Naomi Isabel

6AV1

Profesor: Ary Shared Rosas Carrillo

Bioinformática
22 de octubre de 2026

Índice

1 Introducción **1**

2 Marco teórico **2**
2.1 El problema del plegamiento de proteínas . . . 2
2.2 El modelo HP . . . 2
2.3 Por qué el problema es tan difícil . . . 4
2.3.1 Tamaño del espacio de búsqueda . . . 4
2.3.2 Complejidad computacional . . . 4
2.3.3 Forma del paisaje de energía . . . 4
2.3.4 Cadenas de referencia . . . 5
2.4 Representación de conformaciones . . . 5
2.5 Optimización por enjambre de partículas . . . 5
2.5.1 Coeficientes variables . . . 6
2.5.2 Estabilidad . . . 6
2.6 Diversidad del enjambre . . . 6
2.7 Prueba de Mann-Whitney . . . 7

3 Desarrollo de la práctica **8**
3.1 Cadenas de prueba . . . 8
3.2 Representación de los individuos . . . 8
3.2.1 Direcciones relativas . . . 8
3.2.2 Tres preferencias por paso . . . 9
3.2.3 Decodificación con reparación . . . 9
3.2.4 Función de aptitud . . . 11
3.2.5 Implementación y verificación . . . 12
3.3 Movimiento del enjambre . . . 13
3.4 Estrategia de los coeficientes . . . 13
3.4.1 Curva coseno . . . 13
3.4.2 Recalentamiento por estancamiento . . . 14
3.5 Medidas de diversidad . . . 15
3.6 Algoritmo completo . . . 15
3.7 Ejemplo: evolución de una partícula . . . 16
3.8 Diseño experimental . . . 17

4 Resultados **18**
4.1 Calidad de las soluciones finales . . . 18
4.2 Convergencia del gbest . . . 18
4.3 Diversidad del enjambre . . . 20
4.3.1 Coeficientes usados . . . 22
4.4 Comparación de estrategias de coeficientes . . . 22

5 Conclusiones **24**

Bibliografía **26**

aggedbottom

1 Introducción

Una proteína es una cadena de aminoácidos, y la forma tridimensional que adopta en el agua determina casi todo lo que puede hacer: qué moléculas reconoce, qué reacciones cataliza y con qué otras proteínas se une. Conocer la secuencia de aminoácidos es relativamente sencillo; conocer la forma a la que se pliega es mucho más difícil. Predecir esa forma a partir de la secuencia es uno de los problemas centrales de la bioinformática, y durante décadas fue también uno de los más estudiados en computación, porque el número de formas posibles crece de manera exponencial con la longitud de la cadena.

Para estudiar el problema sin toda la complejidad química de una proteína real, Dill propuso en 1985 el modelo hidrofóbico-polar, o modelo HP [1, 2]. En este modelo cada aminoácido es solo de dos tipos, hidrofóbico (H) o polar (P), la cadena se dibuja sobre una cuadrícula y la energía de una forma depende únicamente de cuántos pares de aminoácidos H quedan juntos. A pesar de ser tan simple, encontrar la forma de menor energía en el modelo HP es un problema NP-completo [3, 4], y se ha convertido en un problema de prueba clásico para algoritmos de búsqueda.

En esta práctica aplicamos la Optimización por Enjambre de Partículas (PSO) al modelo HP en dos dimensiones. A diferencia de las prácticas anteriores, donde las soluciones eran vectores de números reales, aquí una solución es una forma de la cadena en la cuadrícula, que es un objeto discreto y que además no puede cruzarse consigo mismo. Por eso la parte más importante del trabajo fue decidir cómo representar un plegamiento con los números continuos que maneja PSO. Elegimos una representación con direcciones relativas, en la que cada partícula guarda tres valores de preferencia por cada paso de la cadena (girar a la izquierda, seguir de frente o girar a la derecha), y un decodificador que repara los choques de la cadena consigo misma.

También hicimos variar los coeficientes w, c_1 y c_2 durante la ejecución, pero en lugar de una variación lineal usamos una curva coseno a la que se suma un recalentamiento cuando el mejor valor del enjambre deja de mejorar. Probamos el algoritmo en cinco cadenas de referencia de la literatura, de 20 a 64 aminoácidos, con 30 ejecuciones independientes por cadena. En cada iteración registramos el mejor valor encontrado (*gbest*) y tres medidas de diversidad del enjambre (de posición, de velocidad y cognitiva), y reportamos su media y desviación estándar. Por último, comparamos la estrategia de coeficientes elegida contra las curvas lineal, exponencial y coseno sin recalentamiento con la prueba de Mann-Whitney.

2 Marco teórico

2.1 El problema del plegamiento de proteínas

Una proteína se sintetiza como una cadena lineal de aminoácidos unidos por enlaces peptídicos. Cada aminoácido tiene una cadena lateral distinta, y algunas de estas cadenas laterales interactúan bien con el agua (son polares o tienen carga) mientras que otras la evitan (son hidrofóbicas). Al quedar en solución, la cadena se dobla sobre sí misma hasta alcanzar una forma tridimensional estable, llamada estado nativo, y es esa forma la que le permite cumplir su función biológica.

Los experimentos de Anfinsen con la ribonucleasa mostraron que una proteína desnaturalizada recupera por sí sola su forma activa cuando se regresan las condiciones normales. De ahí surgió la llamada hipótesis termodinámica: la información necesaria para plegarse está contenida en la secuencia, y el estado nativo corresponde a la conformación de mínima energía libre [5]. Esta hipótesis convierte el plegamiento en un problema de optimización: dada una secuencia, encontrar la conformación que minimiza una función de energía.

Entre las fuerzas que intervienen, la más importante para el plegamiento global es el efecto hidrofóbico [6, 7]. Los aminoácidos hidrofóbicos tienden a agruparse en el interior de la proteína, lejos del agua, y los polares quedan en la superficie. En muchas proteínas globulares este núcleo hidrofóbico es la característica estructural más clara.

El problema es difícil incluso para la propia proteína si se piensa en términos de búsqueda. Levinthal hizo notar que si una cadena de unos cien aminoácidos probara sus conformaciones una por una, aun a gran velocidad, tardaría más que la edad del universo en encontrar la correcta, y sin embargo las proteínas reales se pliegan en milisegundos o segundos [8]. Esto se conoce como la paradoja de Levinthal, y muestra que la cadena sigue un paisaje de energía que la guía hacia el estado nativo sin probar todas las conformaciones. Para un algoritmo, que no tiene esa guía física, el tamaño del espacio de conformaciones sí es el obstáculo principal.

Hoy existen métodos de aprendizaje profundo, como AlphaFold, que predicen la estructura de muchas proteínas con una precisión cercana a la experimental [9]. Estos métodos aprenden de miles de estructuras conocidas y no resuelven el problema de optimización de forma explícita. Los modelos simplificados como el HP siguen siendo útiles para estudiar el problema de búsqueda en sí mismo y para comparar algoritmos de optimización en condiciones controladas.

2.2 El modelo HP

El modelo hidrofóbico-polar fue propuesto por Dill [1] y estudiado en detalle por Lau y Dill [2]. Conserva solo el efecto hidrofóbico y hace tres simplificaciones:

1. Los 20 aminoácidos se reducen a dos clases: hidrofóbicos (H) y polares (P). Una proteína se describe entonces con una cadena $s = s_1 s_2 \cdots s_n$, con $s_i \in \{H, P\}$.

2. La cadena se coloca sobre una red regular. En esta práctica usamos la red cuadrada en dos dimensiones, así que cada aminoácido ocupa un punto de coordenadas enteras.

3. La energía solo cuenta los contactos entre aminoácidos H.

Una conformación es una asignación de posiciones $\mathbf{p}_1, \ldots, \mathbf{p}_n \in \mathbb{Z}^2$ que cumple dos condiciones. La primera es que aminoácidos consecutivos ocupen puntos vecinos de la red, $\|\mathbf{p}_{i+1} - \mathbf{p}_i\| = 1$. La segunda es que dos aminoácidos no ocupen el mismo punto, $\mathbf{p}_i \neq \mathbf{p}_j$ si $i \neq j$. A un recorrido con estas propiedades se le llama *caminata autoevitante*.

Dos aminoácidos i y j forman un *contacto topológico* si están en puntos vecinos de la red pero no son consecutivos en la cadena ($|i - j| > 1$). La energía de una conformación C es

$$E(C) = \sum_{i<j-1} \varepsilon(s_i, s_j)\, \Delta(i, j), \qquad \varepsilon(s_i, s_j) = \begin{cases} -1 & \text{si } s_i = s_j = H, \\ 0 & \text{en otro caso,} \end{cases} \tag{1}$$

donde $\Delta(i, j) = 1$ si $\mathbf{p}_i$ y $\mathbf{p}_j$ son vecinos en la red y 0 si no lo son. En otras palabras, la energía es menos el número de contactos H-H. El problema consiste en encontrar la conformación C^* que minimiza E, que en este modelo se toma como el estado nativo.

Los pares consecutivos no se cuentan porque siempre están juntos, sin importar cómo se doble la cadena, y por lo tanto no distinguen una conformación de otra. Los contactos con aminoácidos P tampoco aportan energía. La Figura 1 muestra la misma cadena de ocho aminoácidos estirada y plegada: la cadena estirada no tiene contactos H-H, mientras que al doblarla la primera H queda junto a la última y la tercera junto a la sexta, con lo que la energía baja a -2.

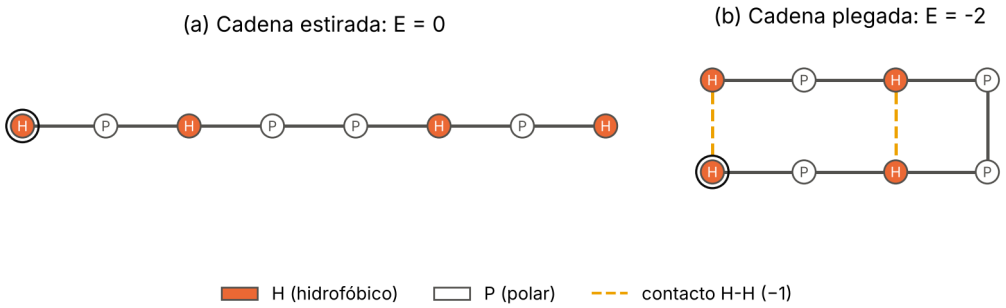


Figura 1: La secuencia HPHPPHPH en dos conformaciones. El círculo negro marca el primer aminoácido. (a) Estirada, sin contactos H-H. (b) Plegada en un rectángulo de 2×4, con dos contactos H-H (líneas punteadas).

En una red cuadrada, cada aminoácido interior de la cadena tiene cuatro vecinos en la red, y dos de ellos ya están ocupados por sus vecinos en la cadena. Por eso un aminoácido interior puede formar a lo más dos contactos, y uno de los extremos a lo más tres. Además, la red cuadrada es bipartita (se puede colorear como un tablero de ajedrez), y dos puntos vecinos siempre tienen colores distintos. Como los aminoácidos de posición par e impar alternan colores a lo largo de la cadena, solo pueden formar contacto un aminoácido en posición par con uno en posición impar. Estas restricciones limitan cuántos contactos puede tener una cadena, pero no dicen cómo conseguirlos.

2.3 Por qué el problema es tan difícil

2.3.1 Tamaño del espacio de búsqueda

El número de conformaciones posibles de una cadena es el número de caminatas autoevitantes de $n-1$ pasos. Para la red cuadrada no se conoce una fórmula cerrada, pero se sabe que crece de forma exponencial,

$$c_m \sim A\,\mu^m\, m^{\gamma-1}, \tag{2}$$

donde c_m es el número de caminatas de m pasos, $\mu \approx 2.638$ es la constante de conectividad de la red cuadrada y $\gamma = 43/32$ [10]. La constante μ es un poco menor que 3 porque, después del primer paso, cada nuevo paso tiene a lo más tres opciones (no puede regresar sobre el anterior), y algunas de ellas chocan con la propia cadena. Para una cadena de 100 aminoácidos, $\mu^{99} \approx 5 \times 10^{41}$, así que la búsqueda exhaustiva es imposible incluso para cadenas de tamaño moderado.

2.3.2 Complejidad computacional

El tamaño del espacio no basta para decir que un problema es difícil: hay problemas con espacios enormes que se resuelven de forma eficiente. En el caso del modelo HP, Crescenzi et al. [3] demostraron que encontrar la conformación de mínima energía en la red cuadrada bidimensional es NP-completo, y Berger y Leighton [4] demostraron lo mismo para la red cúbica en tres dimensiones. Esto significa que no se conoce un algoritmo que garantice encontrar el óptimo en tiempo polinomial, y que encontrarlo para cualquier cadena sería tan difícil como resolver cualquier otro problema de la clase NP. En la práctica se usan métodos exactos para cadenas cortas y metaheurísticas para las largas.

2.3.3 Forma del paisaje de energía

Además del tamaño, la forma de la función objetivo dificulta la búsqueda:

- La energía solo toma valores enteros entre 0 y un mínimo pequeño (por ejemplo, -9 para una cadena de 20 aminoácidos). Millones de conformaciones distintas comparten la misma energía, así que el paisaje está formado por grandes mesetas. Dentro de una meseta, la energía no da ninguna pista sobre hacia dónde moverse.

- Un cambio pequeño en la conformación, como cambiar la dirección de un solo paso, puede mover toda la parte final de la cadena y destruir muchos contactos a la vez. Dos conformaciones parecidas en su descripción pueden tener energías muy distintas.

- Una fracción grande de las descripciones posibles produce cadenas que se cruzan consigo mismas y no son conformaciones válidas. Esta fracción aumenta con la longitud de la cadena (lo medimos en la Sección 3.2).

- La conformación óptima casi nunca es única. Además de las rotaciones y reflexiones, que siempre dan ocho copias equivalentes de cada conformación en la red cuadrada, puede haber conformaciones realmente distintas con la misma energía mínima.

2.3.4 Cadenas de referencia

Para comparar algoritmos se usa un conjunto de secuencias de prueba en dos dimensiones, de 20 a 100 aminoácidos, introducido por Unger y Moult [11] y usado después en muchos trabajos, entre ellos el algoritmo de colonia de hormigas de Shmygelska y Hoos [12]. Para cada secuencia se reporta la mejor energía conocida E^*. En las cadenas cortas esa energía se ha demostrado óptima; en las largas es el mejor valor encontrado hasta ahora por algún método. Los métodos que alcanzan esos valores en cadenas largas suelen combinar una búsqueda global con movimientos locales especializados, como los *pull moves* de Lesh et al. [13], que jalan un segmento de la cadena a una nueva posición sin romperla.

2.4 Representación de conformaciones

Para usar un algoritmo de búsqueda hay que describir cada conformación con una lista de símbolos o números. Como el primer aminoácido puede fijarse en el origen, basta con dar la dirección de cada paso. Krasnogor et al. [14] compararon las dos formas habituales de hacerlo:

- *Codificación absoluta*: cada paso es una dirección fija de la red (arriba, abajo, izquierda o derecha). Hay cuatro opciones por paso, y una de ellas siempre es inválida porque regresa sobre el aminoácido anterior.
- *Codificación relativa*: cada paso se describe respecto a la dirección del paso anterior (girar a la izquierda, seguir de frente o girar a la derecha). Hay tres opciones por paso y ninguna regresa sobre el paso anterior.

Con la codificación relativa el espacio de búsqueda pasa de 4^{n-1} a 3^{n-2} descripciones (el segundo aminoácido se fija también, porque rotar toda la cadena no cambia su energía). Las dos codificaciones también difieren en el efecto de un cambio: con la relativa, cambiar un paso gira todo el resto de la cadena alrededor de ese punto, mientras que con la absoluta lo traslada, y eso cambia qué conformaciones quedan "cerca" unas de otras para el algoritmo [14]. Ninguna de las dos codificaciones impide que la cadena choque consigo misma a más de un paso de distancia, así que cada algoritmo debe decidir qué hacer con esas descripciones inválidas: descartarlas, penalizarlas o repararlas.

2.5 Optimización por enjambre de partículas

PSO fue propuesto por Kennedy y Eberhart en 1995, inspirado en el movimiento coordinado de bandadas de aves [15]. Cada partícula i tiene una posición X_i, que representa una solución, y una velocidad V_i. Guarda además la mejor posición que ha visitado, P_i (*pbest*), y conoce la mejor posición encontrada por todo el enjambre, G (*gbest*). En cada iteración se actualizan con [16]:

$$V_{id}(t+1) = w\,V_{id}(t) + c_1 r_1\,(P_{id} - X_{id}(t)) + c_2 r_2\,(G_d - X_{id}(t))\,, \tag{3}$$

$$X_{id}(t+1) = X_{id}(t) + V_{id}(t+1), \tag{4}$$

donde d es la dimensión y r_1, r_2 son números aleatorios uniformes en $[0, 1]$. El peso de inercia w controla cuánto conserva la partícula su movimiento anterior; un valor

alto favorece la exploración de regiones nuevas y uno bajo la concentra cerca de las soluciones conocidas [17]. El coeficiente cognitivo c_1 atrae a cada partícula hacia su propio *pbest*, y como cada partícula tiene uno distinto, mantiene la variedad del enjambre. El coeficiente social c_2 atrae a todas hacia el mismo *gbest*, lo que acelera la convergencia y reduce la variedad [18].

2.5.1 Coeficientes variables

Al inicio de la búsqueda conviene explorar y al final conviene refinar [19]. Shi y Eberhart propusieron reducir w de forma lineal de 0.9 a 0.4 [20], y Ratnaweera et al. aplicaron la misma idea a los coeficientes de aceleración, disminuyendo c_1 de 2.5 a 0.5 y aumentando c_2 de 0.5 a 2.5 (esquema TVAC) [18]. La variación no tiene que ser lineal: Chatterjee y Siarry [21] propusieron una inercia que decrece de forma no lineal, y otros trabajos ajustan los coeficientes según el estado de la búsqueda [22, 23].

Una curva que se usa mucho para variar un parámetro de forma no lineal es el decaimiento coseno, popularizado en el entrenamiento de redes neuronales por Loshchilov y Hutter, quienes además lo combinan con reinicios periódicos [24]. Para un avance normalizado $s \in [0, 1]$,

$$g(s) = \tfrac{1}{2}\left(1 + \cos(\pi s)\right) \tag{5}$$

va de 1 a 0, cambiando despacio al principio y al final y rápido a la mitad.

2.5.2 Estabilidad

No cualquier combinación de coeficientes produce un enjambre que converge. Poli [25] analizó la media y la varianza de la posición de una partícula y obtuvo que su distribución se estabiliza si

$$c_1 + c_2 < \frac{24\,(1 - w^2)}{7 - 5w}. \tag{6}$$

Si la condición no se cumple, las partículas siguen oscilando con amplitud creciente (limitada en la práctica por la velocidad máxima), lo que equivale a una exploración amplia.

2.6 Diversidad del enjambre

La diversidad mide qué tan dispersas están las partículas y permite observar el paso de la exploración a la explotación [26]. Usamos la distancia promedio de cada vector al centroide:

$$D(\mathbf{v}_1, \ldots, \mathbf{v}_N) = \frac{1}{N}\sum_{i=1}^{N} \left\|\mathbf{v}_i - \bar{\mathbf{v}}\right\|, \qquad \bar{\mathbf{v}} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{v}_i, \tag{7}$$

aplicada a tres conjuntos de vectores. La *diversidad de posición* usa las posiciones actuales X_i e indica qué tan repartido está el enjambre. La *diversidad de velocidad* usa las velocidades V_i; cuando tiende a cero, las partículas se mueven igual o ya no se mueven. La *diversidad cognitiva* usa las mejores posiciones personales P_i e indica qué tan distintos son los "recuerdos" de las partículas; como P_i solo cambia cuando la partícula mejora, esta medida cambia más despacio que las otras dos.

2.7 Prueba de Mann-Whitney

Para decidir si un algoritmo es mejor que otro no basta una ejecución, porque PSO es estocástico. Se ejecuta cada versión varias veces y se comparan las muestras con una prueba no paramétrica, ya que los resultados de las metaheurísticas rara vez siguen una distribución normal [27]. La prueba de Mann-Whitney [28] compara dos muestras independientes: junta los valores de ambas, los ordena, les asigna rangos y calcula el estadístico

$$U = R_1 - \frac{n_1(n_1 + 1)}{2}, \tag{8}$$

donde R_1 es la suma de los rangos de la primera muestra y n_1 su tamaño. Si las dos muestras vienen de la misma distribución, U debe quedar cerca de $n_1 n_2/2$. En la versión unilateral que usamos, la hipótesis alternativa es que los valores de la primera muestra tienden a ser menores que los de la segunda, y la hipótesis nula se rechaza si el valor p es menor que $\alpha = 0.05$. Como la energía del modelo HP solo toma valores enteros, hay muchos empates; la prueba los maneja asignando a los valores empatados el promedio de sus rangos.

3 Desarrollo de la práctica

La implementación está en un cuaderno de Python (`pso_hp_plegamiento.ipynb`) que usa NumPy, Matplotlib y SciPy. La decodificación de las partículas, que es la parte que más se repite, se compila con Numba si está instalado; si no lo está, el mismo código corre en Python normal, solo que más lento. En esta sección describimos las cadenas de prueba, la representación de los individuos, el algoritmo y el diseño experimental.

3.1 Cadenas de prueba

Usamos las secuencias de referencia en dos dimensiones descritas en la Sección 2.3. En el cuaderno están las diez que van de 20 a 100 aminoácidos, y una lista de configuración (`CADENAS_A_CORRER`) indica cuáles se ejecutan. Para esta práctica elegimos las cinco de la Tabla 1, que cubren longitudes de 20 a 64 y permiten ver cómo cambia el comportamiento del algoritmo con el tamaño del problema. La energía mínima conocida E^* de cada una la verificamos con la tabla de Shmygelska y Hoos [12], y el algoritmo la imprime al iniciar.

Cuadro 1: Cadenas usadas en los experimentos. n es la longitud y E^* la energía mínima conocida.

Cadena	n	H	E^*	Secuencia
S1-20	20	10	-9	`HPHPPHHPHPPHPHHPPHPH`
S2-24	24	10	-9	`HHPPHPPHPPHPPHPPHPPHPPHH`
S4-36	36	16	-14	`PPPHHPPHHPPPPPHHHHHHHPPHHPPPPHHPPHPP`
S5-48	48	25	-23	`PPHPPHHPPHHPPPPPHHHHHHHHHHPPPPPP` `HHPPHHPPHPPHHHHH`
S8-64	64	42	-42	`HHHHHHHHHHHHPHPHPPHHPPHHPPHPPHHP` `PHHPPHPPHHPPHHPPHPHPHHHHHHHHHHHH`

3.2 Representación de los individuos

PSO mueve las partículas en un espacio continuo con las Ecuaciones (3) y (4), pero una conformación del modelo HP es un objeto discreto. Necesitábamos entonces una forma de leer un vector de números reales como una conformación, y queríamos que esa lectura cumpliera tres cosas: que cualquier vector diera una conformación válida (o casi siempre), que un cambio pequeño en el vector produjera en general un cambio pequeño en la cadena, y que la representación no favoreciera ninguna dirección sin motivo.

3.2.1 Direcciones relativas

Usamos la codificación relativa de la Sección 2.4. El primer aminoácido se coloca en $(0, 0)$ y el segundo en $(1, 0)$, y a partir de ahí cada paso $k = 1, \ldots, n-2$ decide si la cadena gira a la izquierda (I), sigue de frente (F) o gira a la derecha (D) respecto a la dirección que traía. Fijar los dos primeros aminoácidos elimina las cuatro rotaciones equivalentes de cada conformación, así que el algoritmo no gasta tiempo buscando

la misma forma girada. Las reflexiones no se eliminan: intercambiar todas las I por D produce la imagen en espejo, con la misma energía.

Elegimos la codificación relativa sobre la absoluta por dos razones. La primera es que las tres opciones de cada paso son siempre utilizables respecto al paso anterior, mientras que en la absoluta una de las cuatro regresa sobre el aminoácido anterior y siempre es un choque. La segunda es que el espacio es más pequeño: 3^{n-2} en lugar de 4^{n-1} descripciones.

3.2.2 Tres preferencias por paso

La pregunta siguiente era cómo guardar en números reales una elección entre I, F y D. La opción más directa es usar un número por paso en $[0, 3)$ y redondearlo hacia abajo: 0 es I, 1 es F y 2 es D. La descartamos porque introduce un orden que no existe en el problema. Con esa representación, F queda "entre" I y D: para que una partícula cambie un giro a la izquierda por uno a la derecha tiene que pasar por seguir de frente, y la distancia entre I y D es el doble que entre I y F. En el problema real no hay ninguna razón para que I y D estén más lejos entre sí que de F.

En su lugar, cada partícula guarda tres números por paso, uno para cada opción, que interpretamos como *preferencias*. La posición de una partícula es una matriz

$$X_i = \begin{pmatrix} x_{1,I} & x_{1,F} & x_{1,D} \\ x_{2,I} & x_{2,F} & x_{2,D} \\ \vdots & \vdots & \vdots \\ x_{n-2,I} & x_{n-2,F} & x_{n-2,D} \end{pmatrix}, \qquad x_{k,o} \in [0, 1], \tag{9}$$

que para PSO es simplemente un vector de $D = 3(n-2)$ dimensiones. Para una cadena de 20 aminoácidos son 54 dimensiones y para una de 64 son 186. En cada paso se elige la opción con la preferencia más alta. Así ninguna opción queda entre las otras dos, y las tres son simétricas.

Esta representación también le da un sentido claro a la actualización de PSO. Si el *pbest* de una partícula gira a la derecha en el paso 5, entonces en esa posición la preferencia por D es la más alta, y el término cognitivo de la Ecuación (3) empuja hacia arriba la preferencia por D de la partícula y hacia abajo las otras dos. La partícula no cambia su decisión de golpe: las preferencias se acercan poco a poco y la decisión cambia cuando una supera a otra. Por eso, entre una iteración y la siguiente, la mayoría de los pasos de la cadena se mantiene y solo algunos cambian (lo ilustramos en la Sección 3.7).

3.2.3 Decodificación con reparación

Aunque las tres opciones son válidas respecto al paso anterior, la cadena puede chocar con un aminoácido colocado varios pasos antes. Para medir qué tan seguido ocurre esto, recorrimos todas las descripciones posibles de cadenas cortas y contamos cuántas son caminatas autoevitantes. Para 8 aminoácidos son válidas el 74.5 % (543 de 729), para 12 el 50.9 % (30 073 de 59 049) y para 14 el 41.5 % (220 375 de 531 441). La fracción sigue bajando con la longitud, porque la Ecuación (2) crece como 2.638^m mientras que las descripciones crecen como 3^m. Para las cadenas de 48 y 64 aminoácidos, casi todas las partículas iniciales serían inválidas, y una penalización simple dejaría al enjambre sin información para avanzar.

Por eso el decodificador repara los choques usando el orden de las preferencias. En cada paso:

1. Ordena las tres opciones de mayor a menor preferencia.
2. Prueba la primera. Si la celda de destino está libre, coloca ahí el siguiente aminoácido.
3. Si está ocupada, prueba la segunda opción, y si también lo está, la tercera.
4. Si las tres celdas vecinas están ocupadas, la cadena quedó encerrada y la decodificación se detiene.

Con esta regla la segunda y la tercera preferencia también tienen significado: indican qué hacer si la favorita no se puede. Además, la reparación solo cambia el paso donde ocurrió el choque, así que respeta lo más posible lo que la partícula "quería". La Tabla 2 y la Figura 2 muestran la decodificación completa de una partícula para la cadena `HPHPPHHPHPPH`. En el paso 5, la opción preferida (I) llevaría la cadena a la celda del aminoácido 3, así que el decodificador toma la segunda opción (D).

Cuadro 2: Decodificación de una partícula para la cadena HPHPPHHPHPPH. Cada paso coloca el aminoácido $k + 2$.

Paso k	$x_{k,I}$	$x_{k,F}$	$x_{k,D}$	Orden de preferencia	Elegida
1	**0.84**	0.01	0.23	I, D, F	I
2	0.11	**0.72**	0.31	F, D, I	F
3	**0.57**	0.05	0.29	I, D, F	I
4	**0.95**	0.91	0.35	I, F, D	I
5	**0.68**	0.45	0.51	I, D, F	D (I choca)
6	0.56	**0.77**	0.52	F, I, D	F
7	0.79	0.17	**0.96**	D, I, F	D
8	0.12	**0.76**	0.56	F, D, I	F
9	0.06	0.47	**0.49**	D, F, I	D
10	**0.96**	0.39	0.19	I, F, D	I

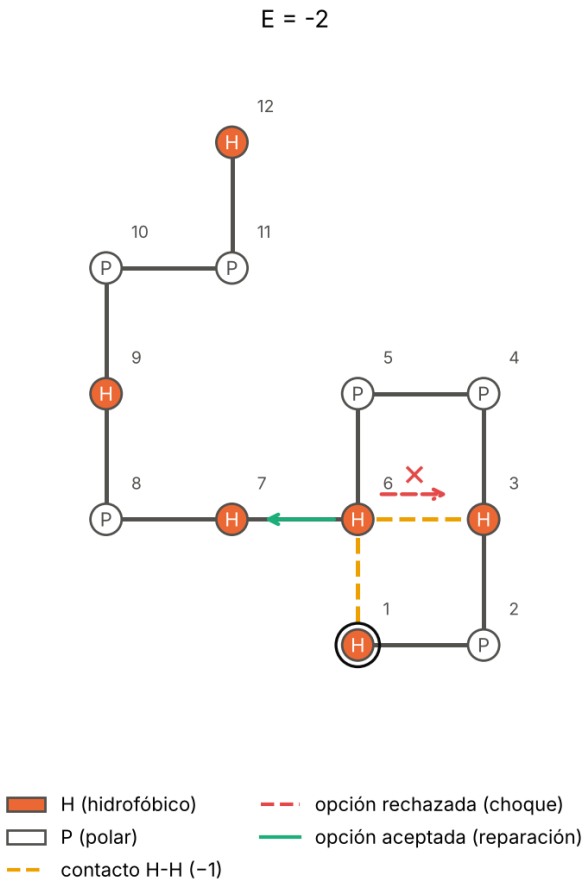


Figura 2: Conformación que resulta de la Tabla 2. Los números indican la posición de cada aminoácido en la cadena. En el paso 5, girar a la izquierda desde el aminoácido 6 chocaría con el aminoácido 3 (flecha punteada), y la cadena gira a la derecha (flecha continua). La energía es -2, por los contactos 1–6 y 3–6.

3.2.4 Función de aptitud

Si la cadena se coloca completa, la aptitud es la energía de la Ecuación (1). Si queda encerrada después de colocar $k < n$ aminoácidos, la aptitud es

$$F = E_{\text{parcial}} + (n+1)(n-k), \tag{10}$$

donde E_{parcial} es la energía de la parte colocada. Esta penalización garantiza que cualquier cadena incompleta sea peor que cualquier cadena completa. En la red cuadrada, los extremos tienen a lo más tres vecinos libres y los aminoácidos interiores dos, y cada contacto ocupa dos de esos lugares, así que una cadena tiene a lo más $n+1$ contactos y $E_{\text{parcial}} \geq -(n+1)$. Con $k < n$, la penalización vale al menos $n+1$ y $F \geq 0$, mientras que toda conformación completa tiene $E \leq 0$. La penalización también crece con el número de aminoácidos que faltan, de modo que entre dos cadenas encerradas es mejor la que llegó más lejos. Con la reparación, estos casos son raros.

3.2.5 Implementación y verificación

El Código 1 muestra el decodificador. Las celdas ocupadas se guardan en una matriz de $(2n+3) \times (2n+3)$ centrada en el origen, suficiente aunque la cadena se estire por completo en cualquier dirección; la matriz se reutiliza entre partículas y solo se limpian las celdas que se usaron. La energía se calcula revisando, para cada aminoácido H, sus cuatro vecinos en la matriz y contando los H que estén más adelante en la cadena y no sean consecutivos.

Código 1: Decodificación de las preferencias de una partícula.

```
@njit
def decodificar(pref, es_h, DX, DY, GIRO, grid, coords):
    n = es_h.shape[0]
    off = n + 1
    coords[0, 0] = 0; coords[0, 1] = 0
    coords[1, 0] = 1; coords[1, 1] = 0
    grid[off, off] = 0
    grid[off + 1, off] = 1
    d, colocados = 0, 2
    for k in range(n - 2):
        orden = np.argsort(-pref[k])     # de mayor a menor
        ok = False
        for t in range(3):
            nd = (d + GIRO[orden[t]]) % 4
            x = coords[colocados - 1, 0] + DX[nd]
            y = coords[colocados - 1, 1] + DY[nd]
            if grid[x + off, y + off] == -1:   # celda libre
                d = nd
                coords[colocados, 0] = x; coords[colocados, 1] = y
                grid[x + off, y + off] = colocados
                colocados += 1
                ok = True
                break
        if not ok:                         # cadena encerrada
            break
    e = 0
    for i in range(colocados):
        if not es_h[i]:
            continue
        for dd in range(4):
            j = grid[coords[i, 0] + DX[dd] + off, coords[i, 1] +
↪ DY[dd] + off]
            if j > i + 1 and es_h[j]:
                e -= 1
    for i in range(colocados):             # limpiar la cuadrícula
        grid[coords[i, 0] + off, coords[i, 1] + off] = -1
    aptitud = e + (n + 1) * (n - colocados)
    return aptitud, e, colocados
```

Antes de optimizar, comprobamos que la energía estuviera bien calculada. Para

la cadena `HPHPPHHPHPPH` recorrimos las $3^{10} = 59\,049$ descripciones, construimos para cada una la partícula equivalente (preferencia 1 en la opción elegida y 0 en las demás) y comparamos la energía del decodificador con la de una función independiente escrita con diccionarios de Python. No hubo ninguna diferencia en las 30 073 conformaciones válidas, y la búsqueda exhaustiva dio el óptimo exacto de esa cadena, $E^* = -5$.

3.3 Movimiento del enjambre

Las posiciones iniciales se generan uniformes en $[0, 1]^D$ y las velocidades uniformes en $[-V_{max}, V_{max}]^D$, con $V_{max} = 0.4$. Después de cada actualización, cada componente de la velocidad se limita a $[-V_{max}, V_{max}]$. Si una preferencia sale de $[0, 1]$, se regresa al borde y su velocidad se invierte y se reduce a la mitad, para que la partícula no siga empujando contra la pared.

Elegimos $V_{max} = 0.4$ en pruebas preliminares con las cadenas S1-20 y S5-48, donde dio mejores resultados que 0.2. Una velocidad alta tiene sentido en esta representación porque lo que importa es el orden de las tres preferencias de cada paso: para cambiar una decisión, una preferencia tiene que rebasar a otra, y con velocidades pequeñas eso tarda muchas iteraciones.

Las mejores posiciones personales se actualizan cuando la nueva aptitud es menor *o igual* que la anterior. Con la comparación estricta, una partícula que encuentra otra conformación con la misma energía no la recuerda. Con la comparación no estricta, el *pbest* puede moverse a lo largo de una meseta del paisaje (Sección 2.3), y eso le da al enjambre una forma de seguir explorando cuando la energía no cambia.

3.4 Estrategia de los coeficientes

3.4.1 Curva coseno

Los tres coeficientes se mueven entre los valores de la Tabla 3 con la curva coseno de la Ecuación (5):

$$w(s) = w_f + (w_i - w_f)\, g(s), \tag{11}$$

$$c_1(s) = c_{1f} + (c_{1i} - c_{1f})\, g(s), \tag{12}$$

$$c_2(s) = c_{2f} + (c_{2i} - c_{2f})\, g(s), \tag{13}$$

donde $s \in [0, 1]$ es la fase de la búsqueda. Los extremos son los del esquema TVAC [18, 20], y con ellos $c_1 + c_2 = 3$ en todo momento: lo que pierde la atracción hacia la memoria propia lo gana la atracción hacia el enjambre.

Cuadro 3: Valores inicial y final de los coeficientes.

Coeficiente	Inicio ($s = 0$)	Final ($s = 1$)
w	0.9	0.4
c_1	2.5	0.5
c_2	0.5	2.5

Elegimos la curva coseno por la forma del paisaje del modelo HP. Como está lleno de mesetas, el enjambre necesita mucho tiempo explorando antes de concentrarse. Una curva exponencial reduce w muy rápido al inicio y deja poco tiempo de

exploración. Una curva lineal reparte el tiempo por igual. La curva coseno cambia despacio al inicio, así que mantiene w y c_1 altos durante una fase larga de exploración; hace la transición rápida a la mitad, y vuelve a cambiar despacio al final, con una fase larga de refinamiento con c_2 alto. La Figura 3 compara las tres curvas.

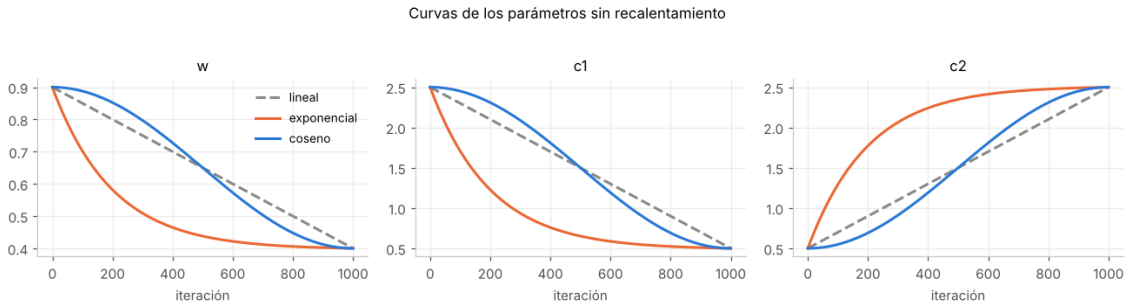


Figura 3: Valores de w, c_1 y c_2 a lo largo de 1000 iteraciones con las curvas lineal, exponencial y coseno, sin recalentamiento.

Según la condición de Poli (Ecuación (6)), con $w = 0.4$ el límite para $c_1 + c_2$ es $24(1-0.16)/(7-2) = 4.03$, mayor que 3, así que al final de la búsqueda las partículas convergen. Con $w = 0.9$ el límite es $24(1 - 0.81)/(7 - 4.5) = 1.82$, menor que 3, y al inicio las partículas no se estabilizan. Esto es intencional: al principio buscamos exploración amplia, y la velocidad máxima evita que las oscilaciones crezcan sin control.

3.4.2 Recalentamiento por estancamiento

Una curva fija no sabe si el enjambre está mejorando o no. Para que los coeficientes reaccionen al estado de la búsqueda, la fase se calcula como

$$s(t) = \frac{t}{T} - b(t), \tag{14}$$

donde t es la iteración, T el total de iteraciones y $b(t) \geq 0$ un retroceso que empieza en cero. En cada iteración b se multiplica por un factor de decaimiento $\delta = 0.98$. Si *gbest* pasa $p = 50$ iteraciones sin mejorar, se suma un impulso $\beta = 0.5$:

$$b \leftarrow \min\left(b + \beta,\ \tfrac{t}{T}\right), \tag{15}$$

y el contador de iteraciones sin mejora vuelve a cero. El límite t/T evita que la fase sea negativa. Al recalentar, la fase retrocede, w y c_1 vuelven a subir y c_2 baja, así que el enjambre recupera capacidad de exploración sin perder sus memorias. Como b decae alrededor de 2 % por iteración, el efecto se reduce a la mitad en unas 34 iteraciones y los coeficientes regresan solos a la tendencia de la curva.

La primera versión de este mecanismo era distinta: en cada estancamiento multiplicaba la fase por 0.5, sin ningún decaimiento. Al graficar los coeficientes usados en las 30 ejecuciones vimos que los estancamientos ocurrían tan seguido que la fase nunca pasaba de alrededor de 0.1, y w se quedaba cerca de 0.88 durante toda la ejecución. En la práctica, la curva coseno no se estaba aplicando. Con la versión de la Ecuación (14) el retroceso es temporal y la curva conserva su forma. En las 30 ejecuciones los resultados de las dos versiones fueron muy parecidos: la media final

de la versión corregida fue mejor en S4-36 (-12.33 contra -12.30), S5-48 (-18.73 contra -18.43) y S8-64 (-29.73 contra -28.90), y un poco peor en S1-20 (-8.67 contra -8.73). Aun así, el hecho de que la primera versión funcionara bien con w casi constante y alto nos dio una pista sobre el problema: en el modelo HP conviene explorar durante mucho tiempo.

3.5 Medidas de diversidad

En cada iteración calculamos las tres medidas de la Ecuación (7). Como las cadenas tienen distinto número de dimensiones, las normalizamos para poder compararlas: la diversidad de posición y la cognitiva se dividen entre $\sqrt{D}$, que es la diagonal del hipercubo $[0,1]^D$, y la de velocidad entre $V_{max}\sqrt{D}$. Con esta normalización, un enjambre con posiciones uniformes al azar tiene diversidad de posición cercana a $\sqrt{1/12} \approx 0.29$ (la desviación estándar de una variable uniforme en $[0,1]$), y velocidades uniformes en $[-V_{max}, V_{max}]$ dan una diversidad de velocidad cercana a $\sqrt{1/3} \approx 0.58$. Esos son los valores de la iteración 0 y sirven como referencia para interpretar las gráficas.

3.6 Algoritmo completo

El Código 2 muestra el ciclo principal de una ejecución.

Código 2: Ciclo principal de PSO (fragmento).

```
for t in range(1, T + 1):
    w, c1, c2 = parametros(s, cfg)
    r1, r2 = rng.random((P, D)), rng.random((P, D))
    V = w * V + c1 * r1 * (pbest - X) + c2 * r2 * (gbest - X)
    np.clip(V, -vmax, vmax, out=V)
    X = X + V
    fuera = (X < 0) | (X > 1)
    V[fuera] *= -0.5
    np.clip(X, 0.0, 1.0, out=X)

    f = evaluar_enjambre(X, es_h, DX, DY, GIRO)
    mejora = f <= pfit                          # acepta empates
    pbest[mejora], pfit[mejora] = X[mejora], f[mejora]
    g = int(np.argmin(pfit))
    if pfit[g] < gfit:
        gbest, gfit = pbest[g].copy(), pfit[g]
        sin_mejora = 0
    else:
        sin_mejora += 1

    b *= cfg["decaimiento"]                     # recalentamiento
    if sin_mejora >= cfg["paciencia"]:
        b = min(b + cfg["impulso"], t / T)
        sin_mejora = 0
    s = t / T - b
```

```
    registrar(t)                                    # gbest y las tres
    ↪ diversidades
```

3.7 Ejemplo: evolución de una partícula

Para ilustrar cómo mejora una partícula, la Figura 4 muestra cuatro momentos hipotéticos de su recorrido sobre la cadena `HPHPPHHPHPPH`, cuyo óptimo es -5. Los números de iteración son ilustrativos, pero las conformaciones y sus energías son reales: las calculamos con el decodificador. Debajo de cada conformación está la decisión ganadora de cada paso.

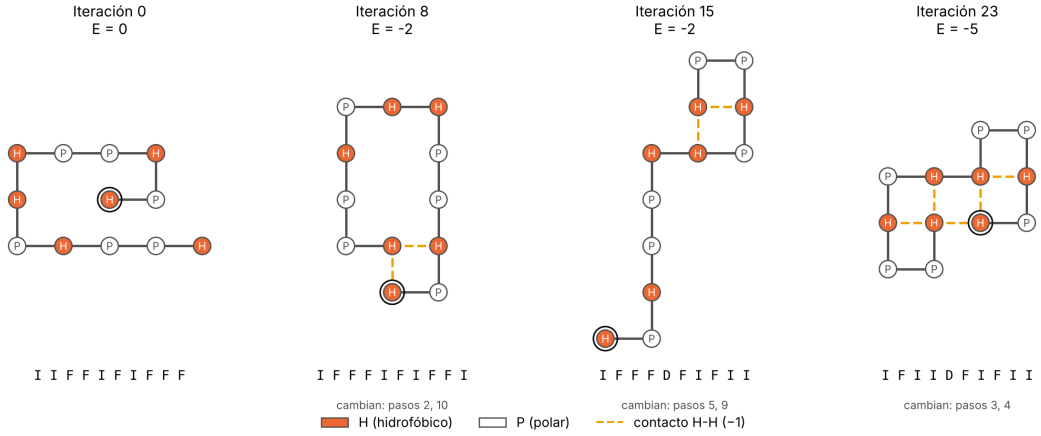


Figura 4: Cuatro momentos de una partícula sobre la cadena HPHPPHHPHPPH. Debajo de cada conformación están las decisiones de los diez pasos (I = izquierda, F = frente, D = derecha) y los pasos que cambiaron respecto al momento anterior.

En la iteración 0 las preferencias son aleatorias. La cadena queda casi estirada, en forma de gancho, y ninguna H toca a otra H que no sea su vecina en la cadena, así que la energía es 0. Esta forma se guarda como su *pbest*, porque es la única que conoce.

Para la iteración 8 las preferencias se movieron hacia el *gbest* del enjambre. Cambiaron solo dos decisiones, los pasos 2 y 10, y eso bastó para doblar la cadena en forma de U: la última H queda junto a la primera y a la tercera. La energía baja a -2 y esta forma se convierte en el nuevo *pbest*.

En la iteración 15 la partícula prueba una forma bastante distinta, después de cambiar los pasos 5 y 9. Ahora los contactos están en otra parte de la cadena, pero la energía sigue en -2. No todos los movimientos mejoran, y que la partícula pruebe formas distintas con la misma energía es parte de cómo el enjambre explora las mesetas. Como aceptamos empates en el *pbest*, esta forma también puede guardarse.

En la iteración 23, al cambiar los pasos 3 y 4, la cadena se cierra sobre sí misma y las H quedan agrupadas en el centro, con cinco contactos. Es la energía mínima de esta cadena. Entre momentos sucesivos solo cambian dos de las diez decisiones, porque las preferencias se mueven de forma gradual y una decisión cambia solo cuando una preferencia rebasa a otra.

3.8 Diseño experimental

Usamos 80 partículas y 1000 iteraciones en todas las cadenas. Elegimos estos valores en pruebas preliminares de 8 ejecuciones: con 50 partículas y $V_{max} = 0.2$ el algoritmo no encontró el óptimo de S1-20 en ninguna, y con 80 partículas y $V_{max} = 0.4$ lo encontró en la mitad de ellas y mejoró la media en S5-48. El tiempo de cómputo no fue un problema: las 30 ejecuciones tardaron entre 21 y 61 segundos por cadena.

Cada cadena se ejecutó 30 veces, con las semillas $2026 + r$ para $r = 0, \ldots, 29$. En cada iteración guardamos *gbest*, las tres medidas de diversidad y los valores de w, c_1 y c_2 realmente usados. Con las 30 ejecuciones calculamos la media y la desviación estándar muestral de cada cantidad en cada iteración. También registramos, para cada ejecución, la iteración en la que alcanzó su valor final de *gbest* y el número de recalentamientos.

Para justificar la estrategia de coeficientes, comparamos cuatro variantes con la misma configuración y las mismas semillas: curva lineal, curva exponencial ($g(s) = (e^{-5s} - e^{-5})/(1 - e^{-5})$), curva coseno sin recalentamiento y curva coseno con recalentamiento. Esta comparación la hicimos en las cadenas S1-20 y S4-36, con 30 ejecuciones por variante. Para cada variante aplicamos la prueba de Mann-Whitney unilateral contra la estrategia elegida, con la hipótesis alternativa de que la estrategia elegida obtiene energías finales menores, usando la función `mannwhitneyu` de SciPy y $\alpha = 0.05$.

4 Resultados

En esta sección reportamos los resultados de las 30 ejecuciones por cadena. Primero presentamos la calidad del *gbest* final y su convergencia, después el comportamiento de las tres medidas de diversidad y de los coeficientes, y al final la comparación entre estrategias de coeficientes.

4.1 Calidad de las soluciones finales

La Tabla 4 resume el *gbest* de la última iteración. Todas las ejecuciones terminaron con una conformación completa, así que el *gbest* es siempre la energía de una conformación válida. El error relativo es $(\bar{E} - E^*)/|E^*|$, donde $\bar{E}$ es la media de las 30 ejecuciones.

Cuadro 4: *gbest* final en 30 ejecuciones por cadena. "Éxitos" cuenta las ejecuciones que alcanzaron la energía mínima conocida, y "Última mejora" es la iteración (media ± desviación estándar) en la que cada ejecución alcanzó su valor final.

Cadena	E^*	Mejor	Peor	Mediana	Media ± desv. est.	Éxitos	Última mejora
S1-20	-9	-9	-8	-9.0	-8.67 ± 0.48	20/30	59 ± 49
S2-24	-9	-9	-7	-8.0	-8.17 ± 0.59	8/30	131 ± 181
S4-36	-14	-14	-11	-12.0	-12.33 ± 0.96	4/30	292 ± 199
S5-48	-23	-21	-15	-19.0	-18.73 ± 1.39	0/30	558 ± 192
S8-64	-42	-33	-24	-30.0	-29.73 ± 2.33	0/30	668 ± 211

En las tres cadenas más cortas el algoritmo alcanzó la energía mínima conocida en al menos una ejecución. En S1-20 lo hizo en 20 de las 30, y en las 10 restantes terminó en -8, a un solo contacto del óptimo. En S2-24 y S4-36 el óptimo apareció en 8 y 4 ejecuciones, y la mediana quedó uno y dos contactos por arriba. En S5-48 y S8-64 ninguna ejecución llegó al óptimo: la mejor quedó a dos contactos en S5-48 (-21 de -23) y a nueve en S8-64 (-33 de -42). El error relativo de la media crece con la longitud: 3.7 % en S1-20, 9.3 % en S2-24, 11.9 % en S4-36, 18.6 % en S5-48 y 29.2 % en S8-64. La desviación estándar también aumenta, de 0.48 a 2.33, lo que indica que en las cadenas largas el resultado depende más de la ejecución.

La columna de la última mejora muestra una diferencia en el tipo de comportamiento. En S1-20 las ejecuciones alcanzaron su valor final, en promedio, en la iteración 59, y el resto de la ejecución no aportó nada. En S5-48 y S8-64 la última mejora ocurrió en promedio después de la iteración 550, y en varias ejecuciones cerca del final. Esto quiere decir que en las cadenas cortas el algoritmo se detiene porque ya encontró lo que podía encontrar, mientras que en las largas se detiene porque se acabaron las iteraciones.

4.2 Convergencia del gbest

La Figura 5 muestra la media y la desviación estándar del *gbest* en cada iteración. En las cinco cadenas la mayor mejora ocurre en las primeras 50 iteraciones, cuando el enjambre pasa de conformaciones aleatorias a conformaciones con algunos contactos. En S1-20 y S2-24 la curva queda casi plana desde la iteración 200. En S4-36 sigue

bajando lentamente hasta la iteración 600. En S5-48 y S8-64 la curva sigue bajando al final de la ejecución, y la pendiente en la segunda mitad es mayor en S8-64, que es la cadena con más margen de mejora.

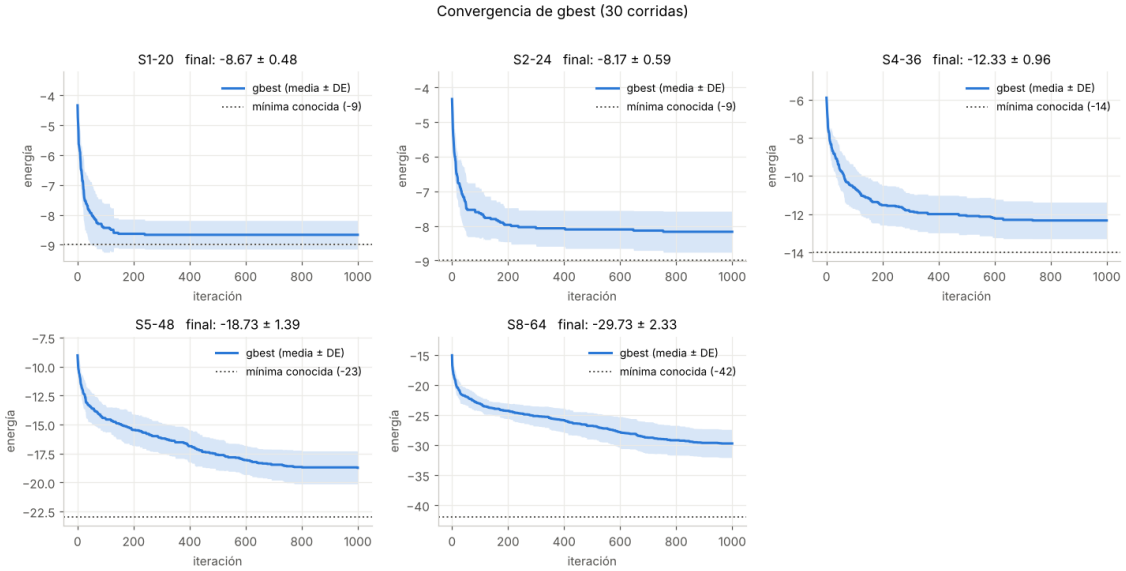


Figura 5: Convergencia del *gbest*: media (línea) y ± una desviación estándar (banda) en 30 ejecuciones. La línea punteada es la energía mínima conocida.

La Figura 6 muestra la mejor conformación encontrada para cada cadena. En todas se observa lo que predice el modelo: las H forman un núcleo compacto en el centro y las P quedan en la orilla o en lazos hacia afuera. En S8-64 algunos segmentos de H quedan fuera del núcleo, en los extremos de la conformación; cada uno de ellos representa contactos que la conformación no aprovecha, y ayuda a entender la distancia de nueve contactos respecto a la mejor energía conocida.

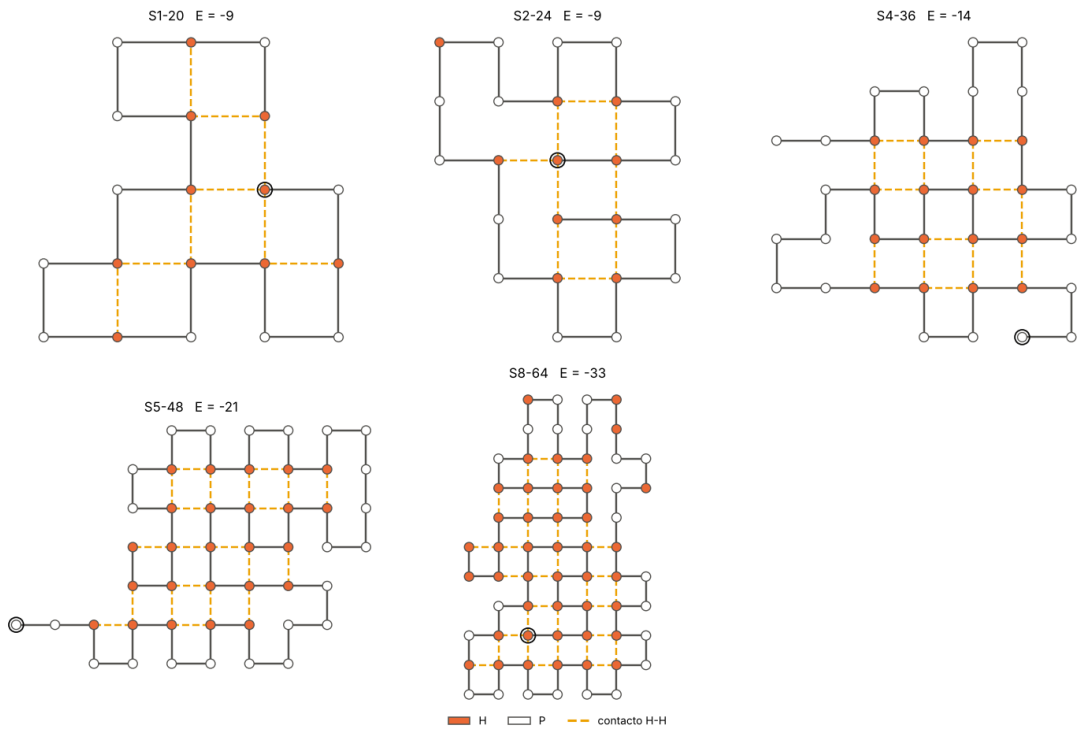


Figura 6: Mejor conformación encontrada en las 30 ejecuciones de cada cadena. El círculo negro marca el primer aminoácido.

4.3 Diversidad del enjambre

La Figura 7 muestra las tres medidas de diversidad para cada cadena, junto con los valores medios de w, c_1 y c_2 que se usaron. La Tabla 5 da los valores en algunas iteraciones.

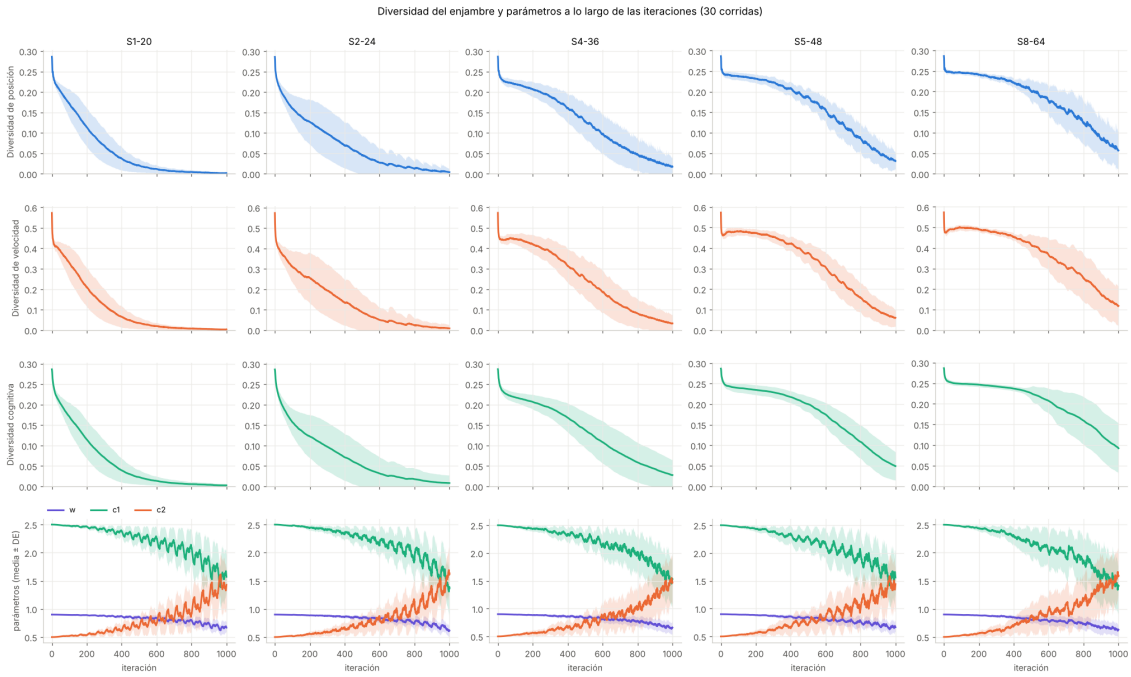


Figura 7: Diversidad de posición, de velocidad y cognitiva (filas 1 a 3) y coeficientes usados (fila 4) para cada cadena: media (línea) y ± una desviación estándar (banda) en 30 ejecuciones.

Cuadro 5: Diversidad normalizada en algunas iteraciones (media ± desviación estándar, 30 ejecuciones).

Medida	Cadena	It. 0	It. 100	It. 250	It. 500	It. 750	It. 1000
Posición	S1-20	0.286 ± 0.002	0.172 ± 0.035	0.089 ± 0.047	0.021 ± 0.016	0.006 ± 0.006	0.002 ± 0.003
	S2-24	0.286 ± 0.002	0.160 ± 0.034	0.113 ± 0.062	0.045 ± 0.045	0.014 ± 0.020	0.004 ± 0.008
	S4-36	0.287 ± 0.001	0.220 ± 0.011	0.198 ± 0.022	0.129 ± 0.044	0.058 ± 0.043	0.018 ± 0.023
	S5-48	0.287 ± 0.001	0.237 ± 0.009	0.228 ± 0.010	0.189 ± 0.023	0.105 ± 0.035	0.031 ± 0.020
	S8-64	0.287 ± 0.001	0.246 ± 0.004	0.237 ± 0.006	0.201 ± 0.026	0.142 ± 0.050	0.057 ± 0.044
Velocidad	S1-20	0.573 ± 0.003	0.329 ± 0.083	0.163 ± 0.091	0.035 ± 0.029	0.009 ± 0.010	0.004 ± 0.005
	S2-24	0.573 ± 0.003	0.311 ± 0.076	0.225 ± 0.132	0.088 ± 0.088	0.025 ± 0.034	0.009 ± 0.017
	S4-36	0.573 ± 0.002	0.445 ± 0.026	0.400 ± 0.053	0.250 ± 0.097	0.103 ± 0.081	0.033 ± 0.043
	S5-48	0.573 ± 0.002	0.481 ± 0.018	0.466 ± 0.021	0.377 ± 0.049	0.198 ± 0.080	0.060 ± 0.043
	S8-64	0.574 ± 0.002	0.498 ± 0.013	0.484 ± 0.012	0.415 ± 0.049	0.291 ± 0.114	0.118 ± 0.095
Cognitiva	S1-20	0.286 ± 0.002	0.171 ± 0.034	0.091 ± 0.047	0.023 ± 0.018	0.007 ± 0.007	0.003 ± 0.004
	S2-24	0.286 ± 0.002	0.157 ± 0.033	0.110 ± 0.059	0.049 ± 0.051	0.019 ± 0.028	0.008 ± 0.019
	S4-36	0.287 ± 0.001	0.218 ± 0.013	0.199 ± 0.021	0.138 ± 0.044	0.070 ± 0.051	0.028 ± 0.038
	S5-48	0.287 ± 0.001	0.239 ± 0.011	0.232 ± 0.012	0.200 ± 0.019	0.126 ± 0.039	0.049 ± 0.034
	S8-64	0.287 ± 0.001	0.249 ± 0.004	0.245 ± 0.005	0.226 ± 0.016	0.172 ± 0.053	0.092 ± 0.060

En la iteración 0 las tres medidas coinciden con los valores de referencia de la Sección 3.5 (0.29 para posición y cognitiva, 0.57 para velocidad), lo que confirma que la normalización es correcta. En todas las cadenas hay una caída brusca en las primeras iteraciones, cuando las partículas abandonan las preferencias aleatorias y se acercan a sus primeras buenas conformaciones.

Después de esa caída, el comportamiento depende de la longitud. En S1-20 y S2-24 la diversidad baja de forma continua y llega casi a cero antes de la iteración

750: el enjambre colapsa sobre una sola conformación, lo que coincide con que el *gbest* deja de mejorar muy pronto. En S4-36, S5-48 y S8-64 las curvas tienen la forma de la curva coseno: se mantienen altas durante cientos de iteraciones, bajan más rápido en la segunda mitad y siguen bajando al final. En S8-64 la diversidad de posición todavía vale 0.057 en la iteración 1000, así que el enjambre no terminó de concentrarse, igual que el *gbest* no terminó de mejorar.

Las tres medidas tienen casi la misma forma, pero hay dos diferencias. La primera es que la diversidad de velocidad baja relativamente más que la de posición en las cadenas cortas (en S1-20, en la iteración 500 la velocidad está en el 6 % de su valor inicial y la posición en el 7 %), lo que indica que las partículas dejan de moverse casi al mismo tiempo que se agrupan. La segunda es que en las cadenas largas la diversidad cognitiva es la que baja más despacio: en S8-64, en la iteración 1000, la cognitiva vale 0.092 y la de posición 0.057. Las mejores posiciones personales solo cambian cuando una partícula mejora o empata, así que el enjambre conserva memorias de regiones distintas aun cuando las posiciones actuales ya se juntaron.

En las cadenas de 36 a 64 aminoácidos, la banda de la desviación estándar es más ancha entre las iteraciones 250 y 750, que es cuando ocurre la transición de la curva coseno y cuando más recalentamientos hay. Cada ejecución hace esa transición en un momento distinto, según cuándo se estanque su *gbest*.

4.3.1 Coeficientes usados

La última fila de la Figura 7 muestra que los coeficientes medios no llegan a sus valores finales. En promedio hubo entre 16 y 19 recalentamientos por ejecución, y cada uno hace retroceder la fase. Por eso, al final de la ejecución, w medio queda alrededor de 0.65 y no en 0.4, y c_1 y c_2 quedan cerca de 1.5 en lugar de 0.5 y 2.5. Las oscilaciones de las curvas medias corresponden a los recalentamientos, y la banda ancha al final indica que algunas ejecuciones estaban recién recalentadas y otras en plena fase de refinamiento. En la práctica, el recalentamiento convirtió la curva coseno en una transición más suave y que no llega a la explotación total, lo que es consistente con la observación de la Sección 3.4 de que este problema favorece explorar durante mucho tiempo.

4.4 Comparación de estrategias de coeficientes

La Tabla 6 y la Figura 8 comparan las cuatro estrategias en S1-20 y S4-36. Los valores p son de la prueba de Mann-Whitney unilateral; un valor menor que 0.05 indica que la estrategia elegida (coseno con recalentamiento) obtiene energías significativamente menores que la estrategia de esa fila.

Cuadro 6: *gbest* final de cada estrategia en 30 ejecuciones, y valor p de la prueba de Mann-Whitney contra la estrategia elegida.

Cadena	Estrategia	Media ± desv. est.	Mejor	Éxitos	p
S1-20 ($E^* = -9$)	Lineal	-8.33 ± 0.76	-9	14/30	0.0384
	Exponencial	-7.63 ± 1.19	-9	9/30	0.0002
	Coseno	-8.63 ± 0.49	-9	19/30	0.3976
	Coseno + recalentamiento	$\mathbf{-8.67 \pm 0.48}$	-9	**20/30**	—
S4-36 ($E^* = -14$)	Lineal	-11.50 ± 1.33	-14	3/30	0.0037
	Exponencial	-10.80 ± 1.47	-14	1/30	< 0.0001
	Coseno	-11.80 ± 1.13	-14	2/30	0.0354
	Coseno + recalentamiento	$\mathbf{-12.33 \pm 0.96}$	-14	**4/30**	—

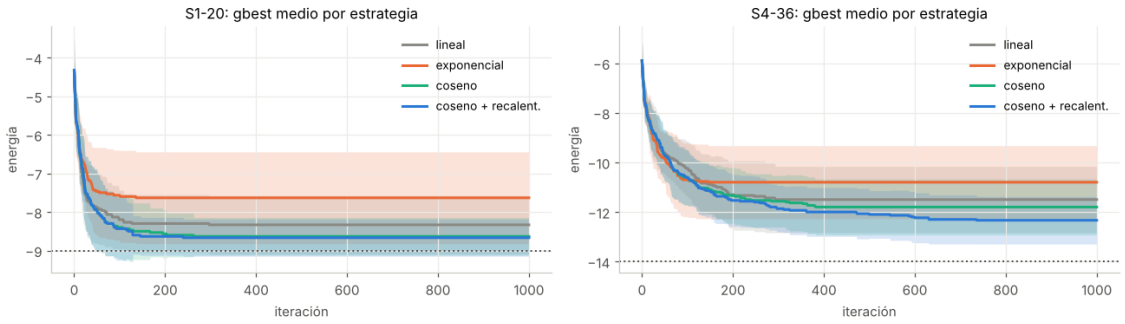


Figura 8: Convergencia media del *gbest* con cada estrategia de coeficientes (30 ejecuciones). La banda es ± una desviación estándar.

La estrategia elegida tuvo la mejor media, la menor desviación estándar y el mayor número de éxitos en las dos cadenas. La diferencia contra la curva lineal y contra la exponencial es significativa en ambas ($p < 0.05$). Contra la curva coseno sin recalentamiento la diferencia es significativa en S4-36 ($p = 0.035$) pero no en S1-20 ($p = 0.398$). En S1-20 las dos versiones coseno tienen casi los mismos resultados (19 y 20 éxitos), porque la cadena es tan corta que el enjambre casi nunca se estanca lejos del óptimo y el recalentamiento tiene poco que hacer.

La curva exponencial fue la peor en las dos cadenas, con la media más alta y la mayor desviación estándar. En la Figura 8 se ve que su curva se aplana antes que las demás: en S4-36 casi no mejora después de la iteración 200. Al reducir w y c_1 muy pronto, el enjambre se concentra en la primera región aceptable y ya no sale de ella. La curva lineal queda en medio, y el orden de las cuatro estrategias (exponencial, lineal, coseno, coseno con recalentamiento) coincide con el orden en que mantienen la exploración durante más tiempo.

5 Conclusiones

En esta práctica aplicamos PSO al problema de encontrar la conformación de mínima energía en el modelo HP bidimensional. El modelo reduce una proteína a una cadena de aminoácidos hidrofóbicos y polares sobre una cuadrícula, y aun así el problema es NP-completo y su espacio de búsqueda crece como 2.638^n. La dificultad principal fue que PSO trabaja con vectores continuos y una conformación es un objeto discreto que no puede cruzarse consigo mismo.

La representación que elegimos resolvió ese problema de forma razonable. Usar direcciones relativas redujo el espacio de búsqueda y eliminó las rotaciones equivalentes. Guardar tres preferencias por paso, en lugar de un solo número redondeado, evitó imponer un orden artificial entre girar a la izquierda, seguir de frente y girar a la derecha, y le dio a la actualización de PSO una interpretación directa: las partículas suben la preferencia por las decisiones de sus mejores conformaciones. El decodificador con reparación fue indispensable. En cadenas de solo 14 aminoácidos, menos de la mitad de las descripciones al azar son conformaciones válidas, y la fracción baja con la longitud; con la reparación, todas las ejecuciones terminaron en conformaciones completas y válidas.

En las cadenas cortas el algoritmo funcionó bien. Encontró la energía mínima conocida en 20 de 30 ejecuciones en S1-20, y al menos una vez en S2-24 y en S4-36. En las cadenas de 48 y 64 aminoácidos no llegó al óptimo: el error relativo de la media fue de 18.6 % y 29.2 %, y la mejor ejecución quedó a dos y nueve contactos de la mejor energía conocida. Las conformaciones encontradas sí tienen la estructura esperada, con un núcleo de H rodeado de P, pero les faltan contactos.

Las medidas de diversidad explicaron esa diferencia mejor que el *gbest* solo. En las cadenas cortas las tres medidas cayeron casi a cero antes de la iteración 750, y en S1-20 el *gbest* dejó de mejorar, en promedio, en la iteración 59: el enjambre colapsó sobre una conformación y el resto de las iteraciones no aportó nada. En las cadenas largas la diversidad seguía bajando al final y la última mejora ocurrió, en promedio, después de la iteración 550. Ahí el enjambre se quedó sin iteraciones antes de terminar de concentrarse, así que el siguiente paso más directo sería darle más tiempo a esas cadenas. La diversidad cognitiva fue la que más tardó en bajar en las cadenas largas, lo que indica que las memorias de las partículas conservan regiones distintas incluso cuando sus posiciones actuales ya se agruparon.

La estrategia de coeficientes con curva coseno y recalentamiento fue la mejor de las cuatro que probamos, con diferencias significativas contra las curvas lineal y exponencial en las dos cadenas, y contra el coseno sin recalentamiento en S4-36. El orden de las estrategias coincidió con cuánto tiempo mantienen la exploración: la exponencial, que la reduce primero, fue la peor. Este resultado, junto con el error que encontramos en la primera versión del recalentamiento (que dejaba w casi fijo en 0.88 y aun así funcionaba bien), nos lleva a la misma conclusión: en un paisaje lleno de mesetas como el del modelo HP, conviene explorar durante mucho tiempo. También aprendimos que conviene graficar también los coeficientes que realmente se usan, porque el error de la primera versión solo se veía en esa gráfica.

Quedan varias mejoras por probar. La más prometedora es combinar PSO con una búsqueda local sobre el *gbest*, por ejemplo con *pull moves* [13], que es lo que hacen los métodos que alcanzan las mejores energías conocidas en cadenas largas. También valdría la pena probar una topología de vecindario en anillo, que tiende

a conservar más diversidad que la topología global, y repetir los experimentos con más iteraciones en las cadenas de 48 aminoácidos o más.

Bibliografía

[1] Dill, K. A. Theory for the folding and stability of globular proteins. *Biochemistry* **1985**, *24*, 1501–1509.

[2] Lau, K. F.; Dill, K. A. A lattice statistical mechanics model of the conformational and sequence spaces of proteins. *Macromolecules* **1989**, *22*, 3986–3997.

[3] Crescenzi, P.; Goldman, D.; Papadimitriou, C.; Piccolboni, A.; Yannakakis, M. On the complexity of protein folding. *Journal of Computational Biology* **1998**, *5*, 423–465.

[4] Berger, B.; Leighton, T. Protein folding in the hydrophobic-hydrophilic (HP) model is NP-complete. *Journal of Computational Biology* **1998**, *5*, 27–40.

[5] Anfinsen, C. B. Principles that govern the folding of protein chains. *Science* **1973**, *181*, 223–230.

[6] Kauzmann, W. Some factors in the interpretation of protein denaturation. *Advances in Protein Chemistry* **1959**, *14*, 1–63.

[7] Dill, K. A. Dominant forces in protein folding. *Biochemistry* **1990**, *29*, 7133–7155.

[8] Levinthal, C. How to fold graciously. Mössbauer Spectroscopy in Biological Systems: Proceedings of a Meeting held at Allerton House, Monticello, Illinois. 1969; pp 22–24.

[9] Jumper, J.; Evans, R.; Pritzel, A.; Green, T.; Figurnov, M., et al. Highly accurate protein structure prediction with AlphaFold. *Nature* **2021**, *596*, 583–589.

[10] Jensen, I. Enumeration of self-avoiding walks on the square lattice. *Journal of Physics A: Mathematical and General* **2004**, *37*, 5503–5524.

[11] Unger, R.; Moult, J. Genetic algorithms for protein folding simulations. *Journal of Molecular Biology* **1993**, *231*, 75–81.

[12] Shmygelska, A.; Hoos, H. H. An ant colony optimisation algorithm for the 2D and 3D hydrophobic polar protein folding problem. *BMC Bioinformatics* **2005**, *6*, 30.

[13] Lesh, N.; Mitzenmacher, M.; Whitesides, S. A complete and effective move set for simplified protein folding. Proceedings of the Seventh Annual International Conference on Research in Computational Molecular Biology (RECOMB '03). 2003; pp 188–195.

[14] Krasnogor, N.; Hart, W. E.; Smith, J.; Pelta, D. A. Protein structure prediction with evolutionary algorithms. Proceedings of the Genetic and Evolutionary Computation Conference (GECCO 1999). San Francisco, CA, USA, 1999; pp 1596–1601.

[15] Kennedy, J.; Eberhart, R. Particle swarm optimization. Proceedings of ICNN'95 - International Conference on Neural Networks. Perth, WA, Australia, 1995; pp 1942–1948.

[16] Shi, Y.; Eberhart, R. A modified particle swarm optimizer. Proceedings of the 1998 IEEE International Conference on Evolutionary Computation. Anchorage, AK, USA, 1998; pp 69–73.

[17] Shi, Y.; Eberhart, R. C. Parameter selection in particle swarm optimization. Evolutionary Programming VII. Berlin, Heidelberg, 1998; pp 591–600.

[18] Ratnaweera, A.; Halgamuge, S. K.; Watson, H. C. Self-organizing hierarchical particle swarm optimizer with time-varying acceleration coefficients. *IEEE Transactions on Evolutionary Computation* **2004**, *8*, 240–255.

[19] Črepinšek, M.; Liu, S.-H.; Mernik, M. Exploration and exploitation in evolutionary algorithms: A survey. *ACM Computing Surveys* **2013**, *45*, 35:1–35:33.

[20] Shi, Y.; Eberhart, R. C. Empirical study of particle swarm optimization. Proceedings of the 1999 Congress on Evolutionary Computation (CEC99). Washington, DC, USA, 1999; pp 1945–1950.

[21] Chatterjee, A.; Siarry, P. Nonlinear inertia weight variation for dynamic adaptation in particle swarm optimization. *Computers & Operations Research* **2006**, *33*, 859–871.

[22] Zhan, Z.-H.; Zhang, J.; Li, Y.; Chung, H. S.-H. Adaptive particle swarm optimization. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* **2009**, *39*, 1362–1381.

[23] Nickabadi, A.; Ebadzadeh, M. M.; Safabakhsh, R. A novel particle swarm optimization algorithm with adaptive inertia weight. *Applied Soft Computing* **2011**, *11*, 3658–3670.

[24] Loshchilov, I.; Hutter, F. SGDR: Stochastic gradient descent with warm restarts. International Conference on Learning Representations (ICLR). 2017.

[25] Poli, R. Mean and variance of the sampling distribution of particle swarm optimizers during stagnation. *IEEE Transactions on Evolutionary Computation* **2009**, *13*, 712–721.

[26] Olorunda, O.; Engelbrecht, A. P. Measuring exploration/exploitation in particle swarms using swarm diversity. 2008 IEEE Congress on Evolutionary Computation. 2008; pp 1128–1134.

[27] Derrac, J.; García, S.; Molina, D.; Herrera, F. A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. *Swarm and Evolutionary Computation* **2011**, *1*, 3–18.

[28] Mann, H. B.; Whitney, D. R. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics* **1947**, *18*, 50–60.